

EXPERIMENT NO.5

Name:Aryan Anil Patankar

Class:D20A

Roll No:38

Batch:B

Aim:Deploying a Voting/Ballot Smart Contract

Theory:-

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal

they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage:**

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type**

Converter help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

□ **Code:**

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal {
        string name; // Changed from bytes32 to string
        uint voteCount;
    }
    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) { // Changed bytes32[] to string[]
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
```

```

        name: proposalNames[i],
        voteCount: 0
    }));
}
}

/**
 * @dev Give 'voter' the right to vote on this ballot. May only be called by
 'chairperson'.
 * @param voter address of voter
 */
function giveRightToVote(address voter) external {
    // If the first argument of `require` evaluates
    // to 'false', execution terminates and all
    // changes to the state and to Ether balances
    // are reverted.
    // This used to consume all gas in old EVM versions, but
    // not anymore.
    // It is often a good idea to use 'require' to check if
    // functions are called correctly.
    // As a second argument, you can also provide an
    // explanation about what went wrong.
    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );
    require(
        !voters[voter].voted, "The
        voter already voted."
    );
    require(voters[voter].weight == 0, "Voter already has the right to vote.");
    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to the voter 'to'.
 * @param to address to which vote is delegated
 */
function delegate(address to) external {
    // assigns reference
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");

    require(to != msg.sender, "Self-delegation is disallowed.");

    // Forward the delegation as long as
    // 'to' also delegated.
    // In general, such loops are very dangerous,
    // because if they run too long, they might
    // need more gas than is available in a block.
    // In this case, the delegation will not be executed,
    // but in other situations, such loops might
    // cause a contract to get "stuck" completely.

```

```

while (voters[to].delegate != address(0)) {
    to = voters[to].delegate;

    // We found a loop in the delegation, not allowed.
    require(to != msg.sender, "Found loop in delegation.");
}

Voter storage delegate_ = voters[to];

// Voters cannot delegate to accounts that cannot vote.
require(delegate_.weight >= 1);

// Since 'sender' is a reference, this
// modifies 'voters[msg.sender]'.
sender.voted = true;
sender.delegate = to;

if (delegate_.voted) {
    // If the delegate already voted,
    // directly add to the number of votes
    proposals[delegate_.vote].voteCount += sender.weight;
} else {
    // If the delegate did not vote yet,
    // add to her weight.
    delegate_.weight += sender.weight;
}
}

/**
 * @dev Give your vote (including votes delegated to you) to proposal
'proposals[proposal].name'.
 * @param proposal index of proposal in the proposals array
 */
function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");
    sender.voted = true;
    sender.vote = proposal;

    // If 'proposal' is out of the range of the array,
    // this will throw automatically and revert all
    // changes.
    proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Computes the winning proposal taking all previous votes into account.
 * @return winningProposal_ index of winning proposal in the proposals array
 */
function winningProposal() public view
    returns (uint winningProposal_)
{
    uint winningVoteCount = 0;

```

```

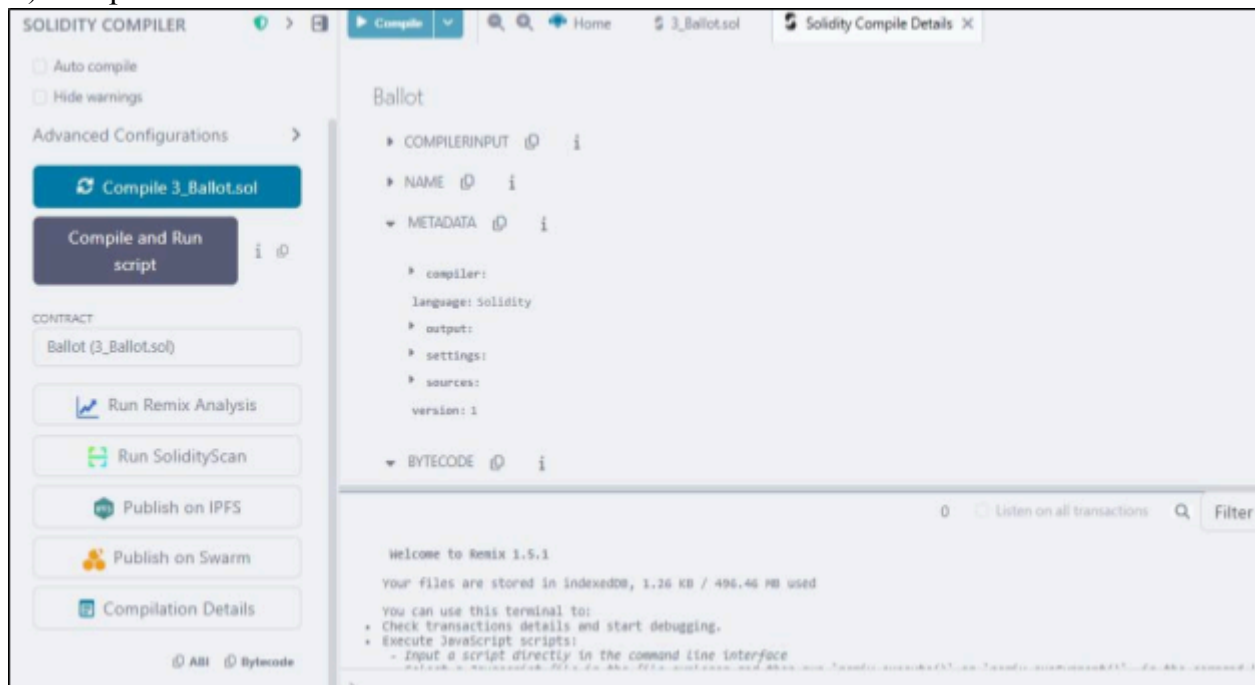
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Calls winningProposal() function to get the index of the winner
    contained in the proposals array and then
     * @return winnerName_ the name of the winner
    */
    function winnerName() external view returns
        (string memory winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }
}

```

Output:-

1) Compile the Ballot.sol Contract



2) Deploying and running of the contract

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The contract 'Ballot' from 'contracts/3_Ballot.sol' is selected. The gas limit is set to 'Estimated Gas' (3000000) and the value is '0 Wei'. The 'Deploy' button is highlighted. Below it, the 'At Address' button is also visible. The 'Transactions recorded' section shows one transaction: 'BALLOT AT 0xD91...39138 (ME)'. The 'Deployed Contracts' section shows the deployed contract at the same address.

The main editor shows the Solidity code for the 'Ballot' contract. The code includes a constructor and a function named 'winnerName()'. The constructor takes an array of proposal names as input. The 'winnerName()' function is a public view function that returns the name of the winning proposal.

```
154 }
155 }
156 }
157
158 /**
159  * @dev Calls winningProposal() function to get the index of the winner contained in the proposals array
160  * @return winnerName_ the name of the winner
161  */
162 //Aryan Patankar D20A 38
163 function winnerName() external view {
164     returns (string memory winnerName_)
165 }
166 winnerName_ = proposals[winningProposal()].name;
```

The 'Explain contract' panel at the bottom shows the transaction details for the deployment. It indicates that the transaction was successful and provides the transaction hash, block hash, block number, and contract address.

3) Loading the proposal candidate's name(string)

The screenshot shows the Remix IDE interface. The 'DEPLOY & RUN TRANSACTIONS' panel is active. The contract 'Ballot' is selected. The 'Deploy' button is highlighted. Below it, the 'At Address' button is also visible. The 'Transactions recorded' section shows one transaction: 'BALLOT AT 0xD91...39138 (ME)'. The 'Deployed Contracts' section shows the deployed contract at the same address.

The main editor shows the Solidity code for the 'Ballot' contract. The code includes a constructor and a function named 'winningProposal()'. The constructor takes an array of proposal names as input. The 'winningProposal()' function is a public view function that returns the index of the winning proposal.

```
142 /**
143  * @dev Computes the winning proposal taking all previous votes into account.
144  * @return winningProposal_ index of winning proposal in the proposals array
145  */
146 function winningProposal() public view {
147     returns (uint winningProposal_)
148 }
149
150 uint winningVoteCount = 0;
151 for (uint p = 0; p < proposals.length; p++) {
152     if (proposals[p].voteCount > winningVoteCount) {
153         winningVoteCount = proposals[p].voteCount;
154         winningProposal_ = p;
155     }
156 }
```

The 'Explain contract' panel at the bottom shows the transaction details for the execution of the 'winningProposal()' function. It indicates that the transaction was successful and provides the transaction hash, block hash, block number, and contract address. The 'decoded input' section shows the input parameters for the function call, including the proposal names: 'Aryan Patankar', 'Vaishnav', and 'Niraj'.

4) Viewing the details of the Proposal Candidate

▼ BALLOT AT 0XD91...39138 (ME)   

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

address voter

▼

vote

uint256 proposal

▼

chairperson

0: address: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4

proposals

proposals - call

▼

0: string: name Aryan Patankar

1: uint256: voteCount 0

voters

address

▼

winnerName

winningProposal

Low level interactions 

CALLDATA

Transact

5) Giving the right to an account other than and by the chairman

The screenshot shows a web interface for a blockchain application. On the left, there's a 'DEPLOY & RUN TRANSACTIONS' section. Under 'Deployed Contracts', the 'BALLOT AT 0xD91...39138 (MEMORY)' contract is selected. The 'giveRightToVote' function is highlighted, with the 'address to' field set to '0xA8483F64D9C6d1EcF9b849Aa677dD33'. The 'vote' field is set to 'uint256 proposal'. The 'chairperson' button is visible. Below, the 'address' field shows '0x5B38Da6a701c56854dCfC803FcB875F56beddC4' and the 'string: name' is 'Aryan Patankar'. The 'uint256: voteCount' is '0'. The 'witness' field is set to 'address'. On the right, the 'Solidity Compile Details' section shows the compiled code for the 'winningProposal' function. Below that, the 'Explain contract' section shows the transaction details: 'status: 1 Transaction mined and execution succeed', 'transaction hash: 0x6c939ce8f0b5336f88eda73f6aa691ecbe00c8c0c6821958f8bcefb8556a6', 'block hash: 0x576c15c9fa967005463781d88c27c3e080666f4d61384128566674a594c85c25', 'block number: 2', 'from: 0x5B38Da6a701c56854dCfC803Fc875F56beddC4', 'to: Ballot.giveRightToVote(address) 0xd9145CCF520386f254917e481a844e9943f39138', 'transaction cost: 48642 gas', 'execution cost: 27268 gas', 'output: 0x', and 'decoded input: { "address voter": "0xA8483F64D9C6d1EcF9b849Aa677dD3315835b32" }'.

6) Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.

The screenshot shows the same web interface as before, but now the 'vote' field is set to '0'. The 'chairperson' button is still visible. Below, the 'address' field shows '0x5B38Da6a701c56854dCfC803FcB875F56beddC4' and the 'string: name' is 'Aryan Patankar'. The 'uint256: voteCount' is '0'. The 'witness' field is set to 'address'. On the right, the 'Solidity Compile Details' section shows the compiled code for the 'winningProposal' function. Below that, the 'Explain contract' section shows the transaction details: 'status: 1 Transaction mined and execution succeed', 'transaction hash: 0xc688f36854ab5744f875a3680b4f4c7c2736075c0a78028a9a2d2138', 'block hash: 0x0889a612d136a286d93a58808878572864e5b21c0957089a9f928f98d', 'block number: 3', 'from: 0xA8483F64D9C6d1EcF9b849Aa677dD3315835b32', 'to: Ballot.vote(uint256) 0xd9145CCF520386f254917e481a844e9943f39138', and 'transaction cost: 72984 gas'.

7) We can view the Voter's Information

The screenshot shows a web interface with a left sidebar and a main content area. The sidebar contains a 'voters' tab, a 'voters - call' button, and a dropdown menu. Below these are input fields for 'uint256: weight 1', 'bool: voted true', 'address: delegate 0x00', and 'uint256: vote 0'. There are buttons for 'winnerName' and 'winningProposal'. A 'Low level interactions' section shows 'CALLDATA' and a 'Transact' button. The main content area displays transaction details for a transaction to 'Ballot.vote pending ...'. The details include a green checkmark, a status of 'Transaction mined and execution succeed', a transaction hash, block hash, block number, fee, to address, transaction cost, execution cost, and output.

8) Selecting the account of the delegator and then writing the address of the voter to be delegated to.

The screenshot shows the Remix IDE interface. The left sidebar contains a 'DEPLOY & RUN TRANSACTIONS' section with a 'Remix VM (Cancun)' environment, an 'ACCOUNT' dropdown, a 'GAS LIMIT' section with 'Estimated Gas' and 'Custom' options, a 'VALUE' section, and a 'CONTRACT' section. The main content area shows a Solidity code editor with a function 'winningProposal()' and a transaction details panel. The transaction details panel shows a transaction to 'Ballot.delegate(address)' with a transaction cost, execution cost, output, decoded input, decoded output, logs, and raw logs.

DEPLOY & RUN TRANSACTIONS

CONTRACT

Ballot - contracts/3_Ballot.sol

evm version: osaka

Deploy [Aryan Patankar, "Vaishnav", "Ning"]

At Address Load contract from Address

Transactions recorded 10

Deployed Contracts 1

BALLOT AT 0XD91...39138 (MEMORY)

Balances 0 ETH

delegate 0xA6348375A9C6d1E6F9b349Aa677dD33

giveRightToVote 0x48209936c481177ec7E8F571ceCaE6A9e2

vote vote - transact (not payable)

chairperson

142 /**

143 * @dev Computes the winning proposal taking all previous votes into account.

144 * @return winningProposal_index of winning proposal in the proposals array

145 */

146 function winningProposal() public view infinite gas

Explain contract

block number 7

from 0x48209936c481177ec7E8F571ceCaE6A9e2

to Ballot.delegate(address) 0x09145CCF52D386F254917e481e84de9943F39138

transaction cost 61199 gas

execution cost 39767 gas

output 0x

decoded input { "address to": "0xA6348375A9C6d1E6F9b349Aa677dD33"

decoded output {}

logs {}

raw logs {}

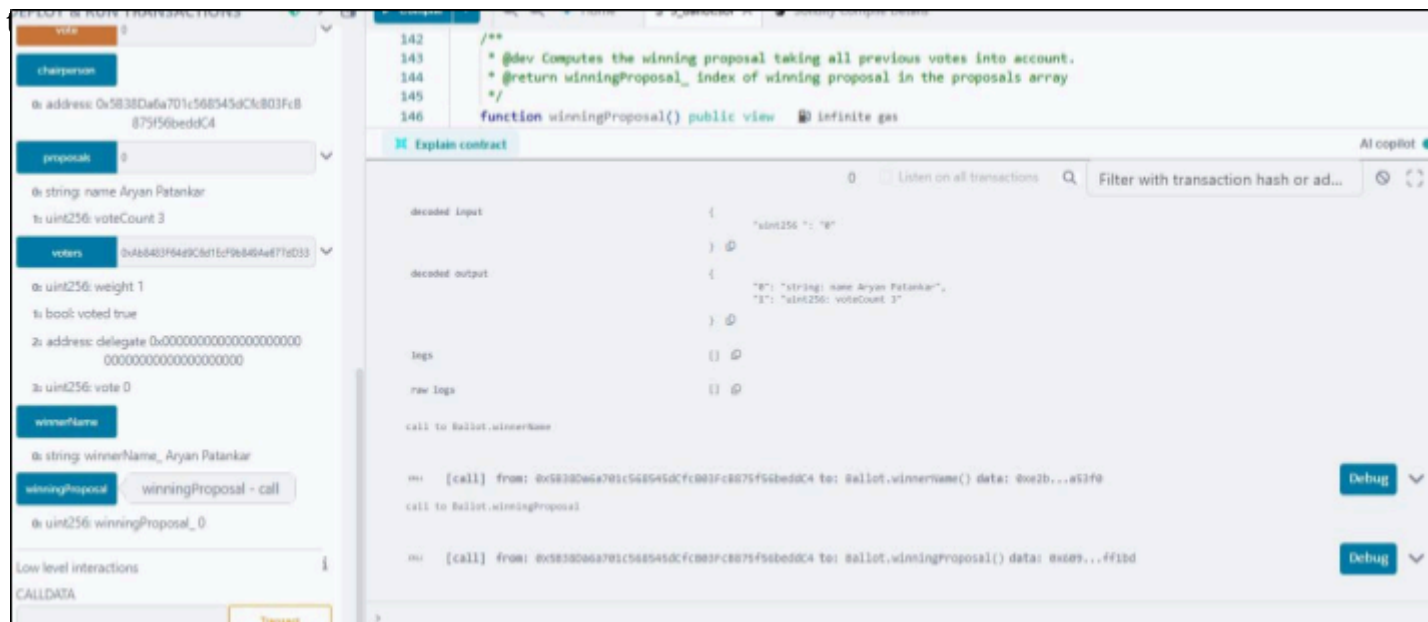
9) Proposal Candidate's Information before giving the Delegate's vote

proposals 0

0: string: name Aryan Patankar

1: uint256: voteCount 3

10) In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.